

Actividad final integradora — Servicios REST

Equipo 9 — Core · Desarrollo de Aplicaciones II

Octubre 2026

Contents

Actividad final integradora — Servicios REST	1
El escenario	1
1. Catálogo de endpoints	2
2. Contrato OpenAPI	4
3. Ejemplos JSON	4
4. Diagrama de secuencia	6
5. Decisiones justificadas	8

Actividad final integradora — Servicios REST

Módulo 9 — Core. Clase 10, Desarrollo de Aplicaciones II.

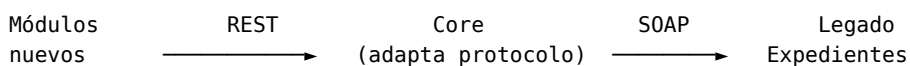
#	Entregable	Dónde
1	Catálogo de endpoints	§1 — 7 operaciones
2	Contrato OpenAPI	openapi-expedientes.yaml — 7 operaciones, todas con re- spuesta de error
3	Ejemplos JSON	§3 — éxito y error en Problem Details
4	Diagrama de secuencia	§4
5	Decisiones justificadas	§5

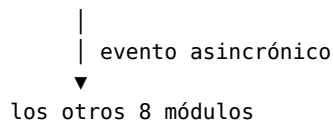
Para entregar: entrega-servicios-rest.pdf o entrega-servicios-rest.docx. Se regeneran desde este mismo archivo con `./build.sh` (necesita pandoc y typst; el diagrama, rsvg-convert).

El escenario

La Municipalidad tiene un **Sistema Central de Expedientes** anterior a la plataforma. Habla **SOAP**, nadie lo puede reescribir, y es el **único autorizado a asignar números oficiales** de expediente (EXP-2026-0001234).

El Core expone una **fachada REST** sobre ese legado:





Por qué le toca al Core: es el módulo de integración. Adaptar un protocolo legado al bus de eventos es exactamente su trabajo, y no rompe la regla de que *el Core no implementa reglas de negocio de las demás áreas*: traduce protocolos, no decide nada del trámite.

1. Catálogo de endpoints

Base: <https://core.muni.uade.edu.ar/api/v1>. Todos requieren Authorization: Bearer <token>.

1. Dar de alta un expediente

POST /expedientes

Headers	Idempotency-Key: <uuid> (obligatorio) , Content-Type: application/json	
Request	tipoTramite, moduloOrigen, solicitante{tipoDocumento, numeroDocumento, nombreCompleto}, asunto, areaResponsable, referenciaExterna (opcional)	
Código	Cuándo	Respuesta
201 Created	El legado numeró dentro de los 5 s	Expediente + header Location
202 Accepted	El legado no respondió en 5 s	SolicitudEnCurso + Location + Retry-After
400 Bad Request	JSON mal formado, o falta Idempotency-Key	Problem Details
401 Unauthorized	Token ausente o inválido	Problem Details
403 Forbidden	El moduloOrigen no coincide con el token	Problem Details
409 Conflict	La Idempotency-Key ya se usó con otro cuerpo	Problem Details
422 Unprocessable	El cuerpo no cumple el contrato	Problem Details + errors[]
503 Service Unavailable	Legado caído y circuit breaker abierto	Problem Details + Retry-After

2. Listar expedientes

GET /expedientes?areaResponsable=obras&estado=EN_TRAMITE&page=1&size=25

Código	Respuesta
200 OK	{items[], total, page, size, pages}
401	Problem Details

3. Consultar un expediente

GET /expedientes/{numero}

Código	Respuesta
200 OK	Expediente
401 · 404	Problem Details

4. Cambiar el estado

PATCH /expedientes/{numero}

Request: {estado, motivo}. El motivo es obligatorio y queda en la auditoría.

Código	Cuándo
200 OK	Estado actualizado
401 · 404	Problem Details
409 Conflict	La transición no es válida (ej. ARCHIVADO → EN_TRAMITE)

5. Registrar una actuación

POST /expedientes/{numero}/actuaciones

Headers: Idempotency-Key. Request: {tipo, descripcion, areaInterviniente}.

Código	Respuesta
201 Created	Actuacion
401 · 404 · 422	Problem Details

6. Listar las actuaciones

GET /expedientes/{numero}/actuaciones

Código	Respuesta
200 OK	Actuacion[], en orden cronológico
401 · 404	Problem Details

7. Seguir una numeración diferida

GET /expedientes/solicitudes/{solicitudId}

Es el recurso al que apunta el Location de un 202.

Código	Cuándo
200 OK	SolicitudEnCurso. Si estado es COMPLETADA, trae el expediente
303 See Other	Ya se completó; redirige al expediente definitivo
401 · 404	Problem Details

2. Contrato OpenAPI

openapi-expedientes.yaml — OpenAPI 3.0.3.

- 4 paths, **7 operaciones**, 9 schemas
- **Todas** las operaciones declaran respuestas de error con `application/problem+json`
- Las 18 referencias `$ref` resuelven (validado)

Se puede pegar en editor.swagger.io para verlo renderizado.

3. Ejemplos JSON

3.1 — Request exitoso

```
POST /api/v1/expedientes HTTP/1.1
Host: core.muni.uade.edu.ar
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
Content-Type: application/json
Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7

{
  "tipoTramite": "RECLAMO_INFRAESTRUCTURA",
  "moduloOrigen": "atencion-ciudadana",
  "referenciaExterna": "TK-2026-00184",
  "solicitante": {
    "tipoDocumento": "DNI",
    "numeroDocumento": "30123456",
    "nombreCompleto": "Ana Perez"
  },
  "asunto": "Bache en Av. Rivadavia 4500",
  "areaResponsable": "obras"
}
```

3.2 — Response exitoso (el legado respondió a tiempo)

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/expedientes/EXP-2026-0001234
X-Trace-Id: 9f2a4c7e-1b33-4d8a-9f10-55ab20c3de91

{
  "numero": "EXP-2026-0001234",
  "estado": "INICIADO",
  "tipoTramite": "RECLAMO_INFRAESTRUCTURA",
  "moduloOrigen": "atencion-ciudadana",
```

```

"referenciaExterna": "TK-2026-00184",
"solicitante": {
  "tipoDocumento": "DNI",
  "numeroDocumento": "30123456",
  "nombreCompleto": "Ana Perez"
},
"asunto": "Bache en Av. Rivadavia 4500",
"areaResponsable": "obras",
"creadoEn": "2026-10-06T14:32:10.482-03:00",
"numeradoEn": "2026-10-06T14:32:12.118-03:00",
"correlationId": "8a1f0c22-5d3e-4b77-9c10-6e2b4a90f3d5"
}

```

3.3 — Error en Problem Details (RFC 7807)

Mismo request, pero con numeroDocumento de 4 dígitos y sin asunto:

HTTP/1.1 422 Unprocessable Entity

Content-Type: application/problem+json

X-Trace-Id: 9f2a4c7e-1b33-4d8a-9f10-55ab20c3de91

```

{
  "type": "https://core.muni.uade.edu.ar/problems/validacion-fallida",
  "title": "La solicitud tiene campos inválidos",
  "status": 422,
  "detail": "2 campos no cumplen el contrato.",
  "instance": "/api/v1/expedientes",
  "traceId": "9f2a4c7e-1b33-4d8a-9f10-55ab20c3de91",
  "errors": [
    {
      "campo": "solicitante.numeroDocumento",
      "mensaje": "Debe tener entre 7 y 11 dígitos.",
      "restriccion": "pattern"
    },
    {
      "campo": "asunto",
      "mensaje": "Es un campo obligatorio y no vino en el cuerpo.",
      "restriccion": "required"
    }
  ]
}

```

Los cinco campos de RFC 7807:

Campo	Para qué
type	URI que identifica la clase de problema. Es la clave de desambiguación: dos errores distintos con el mismo status tienen type distinto.
title	Resumen legible, igual para todas las ocurrencias del tipo.
status	El código HTTP, repetido en el cuerpo para cuando se loguea solo el body.
detail	Qué pasó en esta ocurrencia concreta.
instance	URI de la ocurrencia.

traceId y errors son **extensiones nuestras**, que la RFC permite.

3.4 — El caso interesante: el legado no contestó

HTTP/1.1 202 Accepted

Content-Type: application/json

Location: /api/v1/expedientes/solicitudes/4f1c8e22-9a7b-4c31-b8e2-1d9f0a3c7e55

Retry-After: 15

```
{
  "solicitudId": "4f1c8e22-9a7b-4c31-b8e2-1d9f0a3c7e55",
  "estado": "PENDIENTE_NUMERACION",
  "creadaEn": "2026-10-06T14:32:15.900-03:00",
  "intentos": 1,
  "proximoIntentoEn": "2026-10-06T14:32:30.900-03:00",
  "detalle": "El sistema legado no respondió dentro de los 5 segundos. El expediente quedó registrado y se está reintentando la numeración.",
  "expediente": null
}
```

No es un error. El 202 dice “te lo acepté y lo estoy procesando”. El alta ya está persistida: lo único que falta es el número oficial.

4. Diagrama de secuencia

Los tramos **sincrónicos** van en línea llena, los **asincrónicos** en punteada.

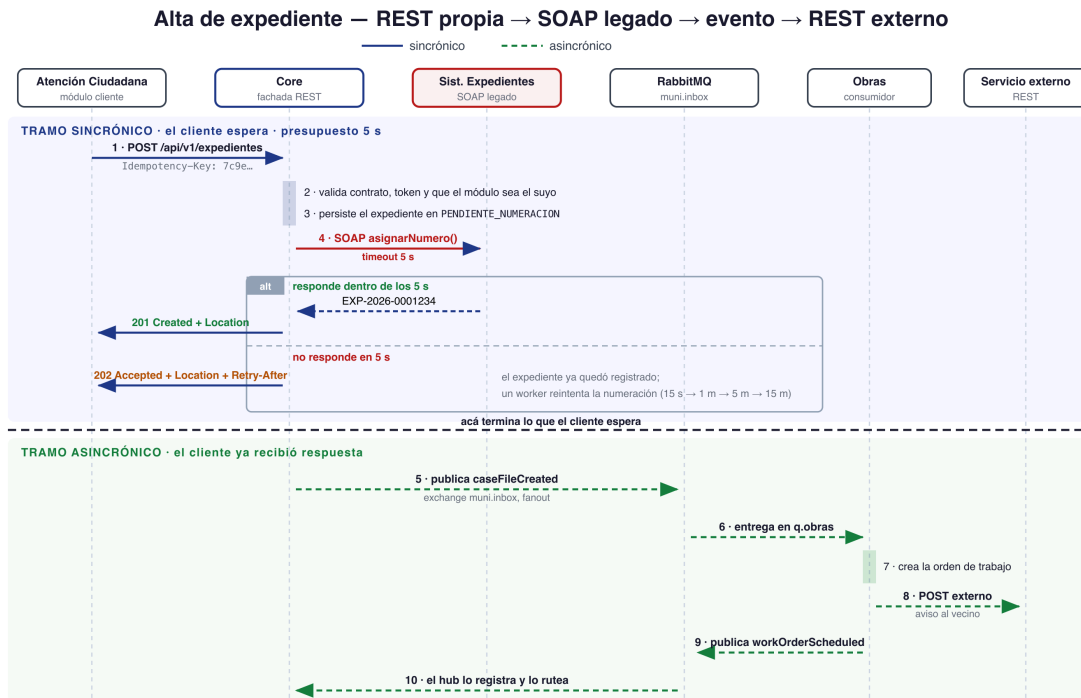


Figure 1: Diagrama de secuencia

Fuente del diagrama (Mermaid)

sequenceDiagram

autonumber

participant M as Atención Ciudadana

participant C as Core (fachada REST)

participant L as Sist. Expedientes (SOAP)

participant Q as RabbitMQ

participant O as Obras

participant X as Servicio externo

rect rgb(235, 240, 255)

Note over M,L: TRAMO SINCRÓNICO – el cliente espera

M->>C: POST /api/v1/expedientes (Idempotency-Key)

C->>C: valida contrato y token

C->>C: persiste PENDIENTE_NUMERACION

C->>L: SOAP asignarNumero() – timeout 5 s

alt Responde dentro de los 5 s

L-->>C: EXP-2026-0001234

C-->>M: 201 Created + Location

else No responde en 5 s

C-->>M: 202 Accepted + Location + Retry-After

Note over C,L: Un worker reintenta con backoff

end

end

rect rgb(240, 255, 240)

Note over C,X: TRAMO ASINCRÓNICO – el cliente ya recibió respuesta

```

C-->>Q: publica caseFileCreated
Q-->>O: entrega en q.obras
O->>O: crea la orden de trabajo
O-->>X: POST al servicio externo
O-->>Q: publica workOrderScheduled
Q-->>C: el hub lo registra y rutea
end

```

La lectura del diagrama:

Tramo	Tipo	Por qué
Módulo → Core	Sincrónico	El cliente necesita saber si se aceptó el alta.
Core → SOAP legado	Sincrónico, con timeout de 5 s	Es el único que asigna el número. Pero el timeout es corto a propósito: no se le traslada al cliente la lentitud del legado.
Core → RabbitMQ	Asincrónico	El cliente ya recibió su respuesta. Que Obras se entere no es su problema.
Obras → REST externo	Asincrónico	Si el externo está caído, el reintento es de Obras; no afecta al alta.

Dónde está el corte: el tramo sincrónico termina cuando el Core responde 201 o 202. Todo lo que sigue es asincrónico, y por eso una caída del servicio externo no puede hacer fallar el alta de un expediente.

5. Decisiones justificadas

5.1 — Idempotencia

Header Idempotency-Key obligatorio en los POST.

El cliente genera un UUID por intento. El Core guarda la respuesta asociada a esa clave por 24 h. Reenviar con la misma clave devuelve **la misma respuesta** sin volver a ejecutar nada.

Por qué es obligatorio y no opcional: porque esta API puede responder 202. Un cliente que recibe 202 o un timeout de red **va a reintentar**. Sin clave de idempotencia, cada reintento crearía un expediente nuevo con un número oficial distinto para el mismo trámite. Eso en un sistema de expedientes es grave: el número es el identificador legal.

Si la misma clave llega con un cuerpo distinto, se responde 409: devolver la respuesta vieja sería mentirle al cliente sobre lo que acaba de pedir.

Es el mismo principio que ya usa el hub de eventos del Core con `eventId`, aplicado a HTTP.

5.2 — Versionado

En la URI: /api/v1/...

Alternativa		Por qué no
Header	Accept: application/vnd.muni.v1+json	Más purista, pero no se ve en el navegador ni en un log, y con 9 equipos integrando, lo que no se ve genera consultas.
	Query param ?version=1	Se pierde al copiar URLs y ensucia el cacheo.

Con la URI el versionado es **visible en cada request**, se puede probar pegando el link en el navegador, y los proxies lo cachean distinto sin configuración.

Qué obliga a subir a v2: sacar un campo de una respuesta, volver obligatorio uno que no lo era, o cambiar el tipo de un campo. Agregar un campo opcional **no** rompe y va en v1.

5.3 — Seguridad

Transporte	HTTPS obligatorio. En producción el 307 de HTTP a HTTPS está deshabilitado: se rechaza, para que un token no viaje en claro ni una vez.
Autenticación	JWT firmado con RS256 . Los módulos lo obtienen con POST /auth/module-token usando su secret de máquina.
Autorización	El token lleva el módulo. Si moduloOrigen del cuerpo no coincide, 403: ningún equipo puede dar de alta expedientes en nombre de otro.
Validación	Los schemas declaran additionalProperties: false. Un campo no previsto se rechaza en lugar de ignorarse.
Rate limiting	Por módulo, no por IP: los 9 están detrás de proxies de su plataforma y la IP no los distingue.
Datos sensibles	El documento del solicitante no se loguea. Los logs llevan el traceId, y el dato se busca por ahí en la base si hace falta.

Por qué RS256 y no HS256: con firma asimétrica los módulos validan el token con la clave pública, sin poder emitir tokens. Con HS256 el secreto de verificación es el mismo que el de firma, y cualquiera que valide podría falsear.

5.4 — Qué responde la API si el SOAP no contesta en 5 s

Pregunta clave de la consigna.

Responde 202 Accepted, con Location al recurso de seguimiento y Retry-After: 15.

El razonamiento

El alta tiene dos partes, y **solo una depende del legado**:

1. Registrar el expediente → lo hace el Core, siempre funciona
2. Asignarle el número oficial → solo lo puede hacer el legado

Cuando el legado no responde, la parte 1 **ya está hecha y persistida**. Devolver un error sería mentir: diría que no se hizo nada cuando en realidad el trámite quedó registrado. El cliente reintentaría y, sin idempotencia, duplicaría.

Entonces: se acepta lo que se pudo hacer, se le da al cliente una forma de seguir el resto, y un worker reintentará la numeración con backoff.

Por qué no las alternativas

Opción	Por qué no
504 Gateway Timeout	Dice “no se hizo nada”, y es falso: el expediente ya está registrado. Induce al cliente a reintentar algo que ya existe.
500	Peor: sugiere un bug nuestro cuando el problema es de un tercero.
Esperar más de 5 s	Traslada la lentitud del legado a los 9 módulos. Con 5 s el cliente tiene una respuesta acotada siempre.
503 siempre	Rechaza el alta cuando sí se pudo registrar. Lo reservamos para cuando el circuit breaker está abierto y ni siquiera se intenta.

Cuándo sí devolvemos error

Si el legado lleva **caído un rato largo** (circuit breaker abierto tras 5 fallos seguidos), ahí sí: 503 con Retry-After: 60. La diferencia con el 202 es real y hay que saberla leer:

- **202** → se intentó, no contestó, **el alta quedó registrada**
- **503** → ni se intentó, **el alta no se hizo**, reintentará más tarde

El ciclo completo

POST /expedientes

└ 202 Accepted, Location: /expedientes/solicitudes/4f1c...

GET /expedientes/solicitudes/4f1c...

└ 200 { estado: "PENDIENTE_NUMERACION", intentos: 2 }

(el worker reintentará: 15 s → 1 m → 5 m → 15 m)

GET /expedientes/solicitudes/4f1c...

└ 303 See Other, Location: /expedientes/EXP-2026-0001234

Agotados los reintentos, la solicitud queda FALLIDA y aparece en el panel del Core para intervención manual — el mismo mecanismo que ya usamos para la DLQ de eventos.

5.5 — Una deuda que reconocemos

Los **45 endpoints que ya tiene el Core** devuelven los errores en un formato propio ({code, message, details, traceId}), **no en Problem Details**. Funciona y es consistente, pero no es el estándar.

La API de Expedientes se diseñó con RFC 7807 desde el arranque. La migración de los endpoints existentes es un cambio acotado —el manejo de errores está centralizado en un único archivo— pero rompe a los clientes que ya parsean el formato actual, así que corresponde hacerla con un salto de versión, no de contrabando.

Plan: migrar en v2, y mientras tanto responder Problem Details cuando el cliente mande Accept: application/problem+json.